Hash Language
Grammar and Semantic Analysis Report

ANTLR-Based Programming Language Design

Author: Ehsan Soveizi
Student ID: 4032262112

Prepared for Automata Theory and Languages Project
June 2026

1. Project Overview

Hash is a small educational programming language designed with ANTLR. The project focuses on defining a readable grammar, generating parse trees, and adding a semantic checking layer in Java. The language uses Persian-style keywords written in Latin form, such as baste for module declaration, biar for import, bebin for function/method definition, jadid for object creation, age/vagarna for conditionals, ta/baraye for loops, and emtehan/gereftar/akhar for exception handling.

The goal of this report is not to explain every grammar rule line by line. Instead, it summarizes the main design idea, the core grammar structure, the semantic checks, and the test cases used to demonstrate the parser and semantic analyzer.

2. General Architecture

Area	Supported Design	Important Semantic Checks
Lexer and Tokens	Defines keywords, operators, literals, identifiers, comments and whitespace handling.	Ensures reserved keywords such as klass, bebin, jadid, age, ta, baraye, bendaz and bechap are tokenized before generic identifiers.
Parser Grammar	Defines program structure, supported statements, class/function syntax, exception blocks, and expression precedence.	The parse tree starts from startState and organizes source code into module/import declarations followed by supported statements.
Semantic Checker	Java listener extending HashBaseListener. It walks the parse tree after parsing.	Uses maps and stacks for variables, functions, classes, return contexts, loop depth and exception types.
Test Suite	Five input files demonstrate the major features of the language.	Each test is paired with a generated parser tree image for visual verification.

3. Grammar Design Summary

The program begins with an optional module declaration, zero or more imports, and then a sequence of supported statements:

```
startState : moduleStatements? importStatements* supportedStatements* EOF;
```

The supported statement layer currently covers:

- typed variable declaration with and without assignment
- defined assignment such as x = expression;
- if/else using age (...) bood { ... } vagarna { ... }
- while and for loops using ta and baraye
- shekan and edame loop-control statements
- switch/case/default using entekhab, halat and digar
- function declarations, return statements and function calls
- class declarations, fields, constructors, methods and object instantiation
- this-style field access using in.field
- print and input statements
- try/catch/finally exception blocks and throw statements using bendaz.

4. Expression Precedence

The expression grammar is organized in layers. This makes the parser reflect the intended priority of operators. Access and calls bind strongly, prefix unary operators are parsed before postfix operators, power is right-associative, and logical operators have lower priority than comparison and arithmetic operators.

```
expression
  -> logicalOrExpression
  -> logicalAndExpression
  -> equalityExpression
  -> comparitioanlExpression
  -> additiveExpression
  -> multiplicativeExpression
  -> powerExpression
  -> postfixExpression
  -> prefixExpression
  -> accessAndCallsExpression
  -> primaryExpression
```

Important design point: x ** y ** z is parsed as x ** (y ** z). Prefix operators such as ++x, --x and !x are checked as unary operations, while postfix forms such as x++ and x-- are attached after the prefix expression.

5. Semantic Checking Design

The semantic checker is responsible for checks that cannot be reliably expressed by the grammar alone.

Area	Supported Design	Important Semantic Checks
Variables	variables: Map<String, String>	Stores declared variable names and their Hash types. Supports input declarations and no-assignment declarations.
Functions	functions: Map<String, FunctionInfo>	Registers function signatures before checking bodies, enabling recursion and forward calls.
Classes	classes: Map<String, ClassInfo>	Stores fields, methods and constructors. Validates object creation, field access and method calls.
Return Context	functionReturnTypes and functionHasReturn stacks	Checks that return statements are inside callables and that returned expression types match the expected return type.
Loops	loopDepth counter	Validates that shekan and edame are used only inside loops.
Exceptions	exceptionTypes set and catch scope snapshots	Validates predefined/custom exception names and limits catch variables to the catch block scope.

6. Type System and Compatibility

Hash currently supports the primitive types adad, ashari, boole, matn and harf, plus khali as a null-like value.

```
Compatible examples:
ashari value = 10;        // adad can be assigned to ashari
```

```
adad n = khali;          // khali is accepted by the semantic checker
matn msg = "hello " + name; // + supports string concatenation

Invalid examples:
adad x = "text";
boole b = 10;
matn s = x++;
```

7. Current Limitations and Simplifications

- The project validates syntax and semantic constraints; it does not execute programs or produce runtime output.
- Switch currently accepts an identifier expression in the grammar, not a full arbitrary expression.
- Standalone x++ or ++x statements require explicit grammar support; postfix/prefix expressions are currently mainly used inside expressions.
- Return checking confirms that a non-hich callable has at least one return statement, but it does not prove that every control-flow path returns.
- General block scoping for if/else and loops is simplified compared with production programming languages.
- Custom exception support currently includes the simple klass MyError; form; full class-bodied exception types can be added as an extension.

8. Test Suite Overview

Area	Supported Design	Important Semantic Checks
Test One	Demonstrates primitive declarations, module/import syntax, assignment, expression precedence, power associativity, prefix/postfix operators, string concatenation, input declaration, null literal, and an intentional type mismatch.	Mostly valid syntax. The line assigning x++ to a matn variable is intended to produce a semantic type error.
Test Two	Demonstrates function declaration, parameter typing, nested if/else with bood, switch-case, void function, return statements, recursive Fibonacci calls, and function-call type checking.	Contains intentional semantic errors: assigning a matn-returning function to adad and passing matn to a function expecting adad.
Test Three	Demonstrates class declaration, fields, constructor, this/in field assignment, typed methods, hich methods, object instantiation with jadid, method calls, object field assignment and field access.	Contains one intentional semantic error: a constructor argument of type matn is passed where adad is expected.
Test Four	Demonstrates custom exception declaration, try/catch/finally blocks, predefined exception types, custom exception types and exception throw syntax.	Includes intentional invalid forms such as throw without parentheses and a case-sensitive keyword typo, useful for showing parser/semantic behavior.
Test Five	Demonstrates while, for, nested control flow, if conditions, shekan, edame, assignment updates and print statements.	Expected to pass semantic checks because loop conditions are boolean and shekan/edame are inside loops.

Appendix - Input Test Files and Parser Trees

The following sections include the submitted input test files and their generated parser tree images in order. Some tests intentionally contain semantic or syntax mistakes to demonstrate the behavior of the parser and semantic checker.

1. Test One - Assignments, Operators, Input and Print

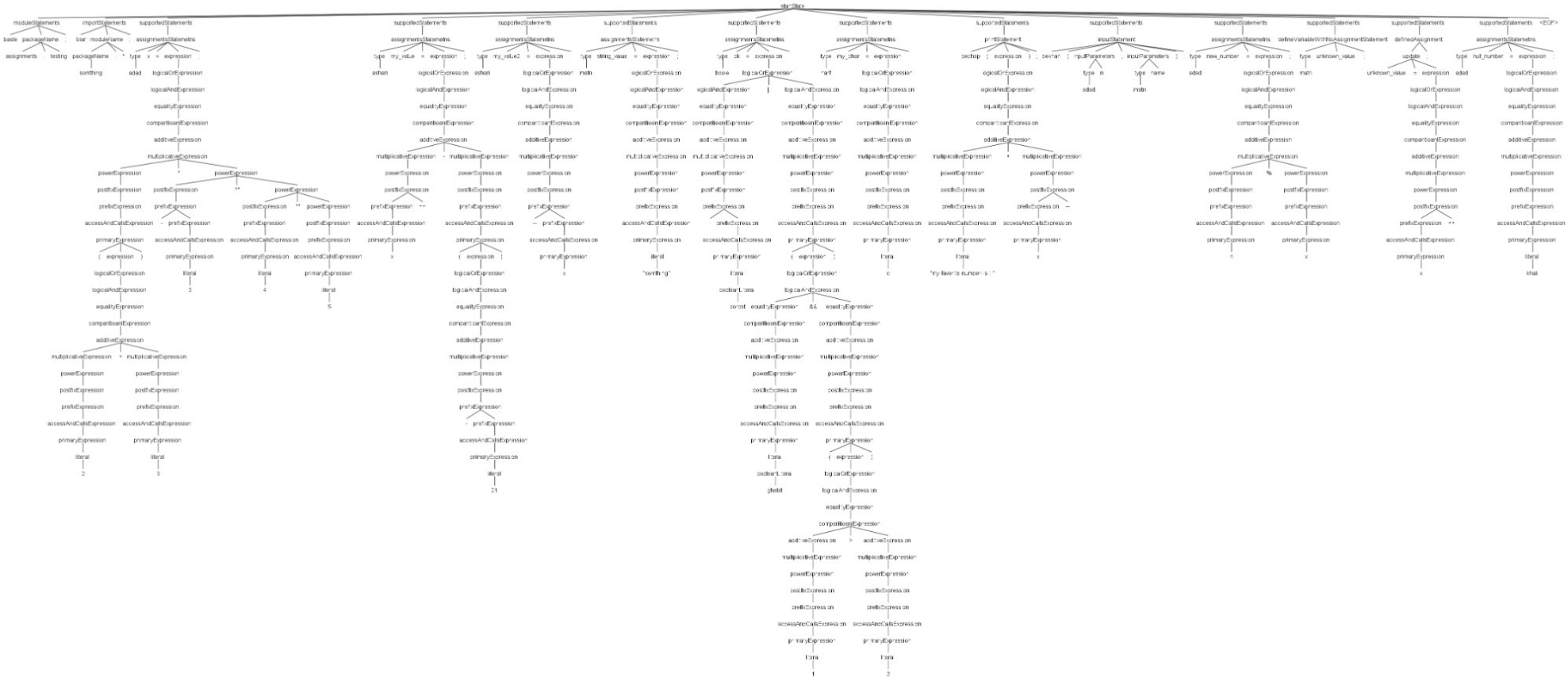
Purpose: Demonstrates primitive declarations, module/import syntax, assignment, expression precedence, power associativity, prefix/postfix operators, string concatenation, input declaration, null literal, and an intentional type mismatch.

Expected behavior: Mostly valid syntax. The line assigning x++ to a matn variable is intended to produce a semantic type error.

Input Test File

```
/* File_name : Test_one
related : assignments priority checking with supported operations
*/

baste assignments.testing;

biar somthing.*;


adad x = (2 + 3) * -3 ** 4 ** 5;
ashari my_value = x++ - (-21);
ashari my_value2 = --x; // ashari type accepts both ashari and adad !
matn string_value = "somthing" ; // must be in "" sign
boole ok = !dorost || (ghalat && (1 > 2)); // can be get values like comparitions and equality checking
harf my_char = 'c'; // must be inside a '' sign

bechap("my favorite number is : " + x--);
bekhan(adad n,matn name);

adad new_number = n % x; // here n is defined and will store to my Hashmap Variables
matn unknown_value; // we can define a value without assignments
unknown_value = x++; // here we get exception because we assign an adad value inside a matn type variable
adad null_number = khali; // khali can be assigned to any proper variable types
```

Parser Tree Image



2. Test Two - Functions, If/Else, Switch and Recursion

Purpose: Demonstrates function declaration, parameter typing, nested if/else with bood, switch-case, void function, return statements, recursive Fibonacci calls, and function-call type checking.

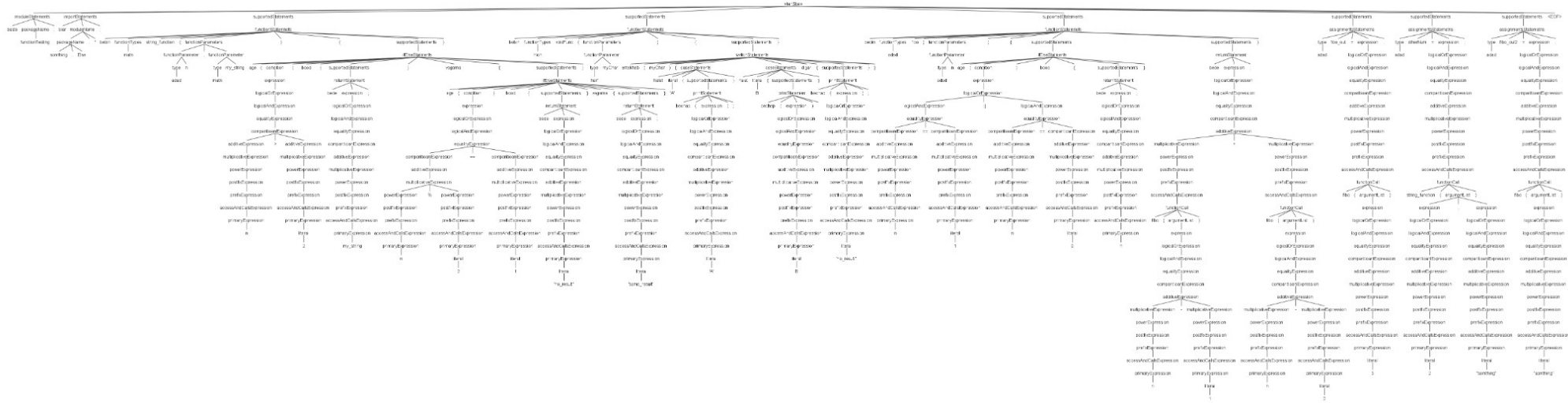
Expected behavior: Contains intentional semantic errors: assigning a matn-returning function to adad and passing matn to a function expecting adad.

Input Test File

```
/*File_name : test_two.txt
related to : function declaretion and ifElse testing with swichCase testing
*/

baste functionTesting;

biar somthing.Else.*;

bebin matn string_function(adad n,matn my_string){
   age(n > 2) bood {
       bede my_string;
   }
   vagarna{
     age(n % 2 == 1) bood {bede "no_result";}
     vagarna{bede "some_result";}
     }
   }

bebin hich voidFunc(harf myChar){
    entekhab(myChar){
        halat 'A'{
          bechap('A');
        }
        halat 'B'{
           bechap('B');
        }
        digar{
          bechap("no_result");
        }
       }
}

bebin adad fibo(adad n){
   age(n == 1 || n == 0) bood{bede n;}
     bede fibo(n - 1) + fibo(n - 2);
   }

adad fibo_out = fibo(3);
```

```
adad otherNum = string_function(2,"somthing"); // we will get exception here

adad fibo_out2 = fibo("somthing"); // it's is wrong as well
```

Parser Tree Image



3. Test Three - Classes, Constructors and Object Usage

Purpose: Demonstrates class declaration, fields, constructor, this/in field assignment, typed methods, hich methods, object instantiation with jadid, method calls, object field assignment and field access.

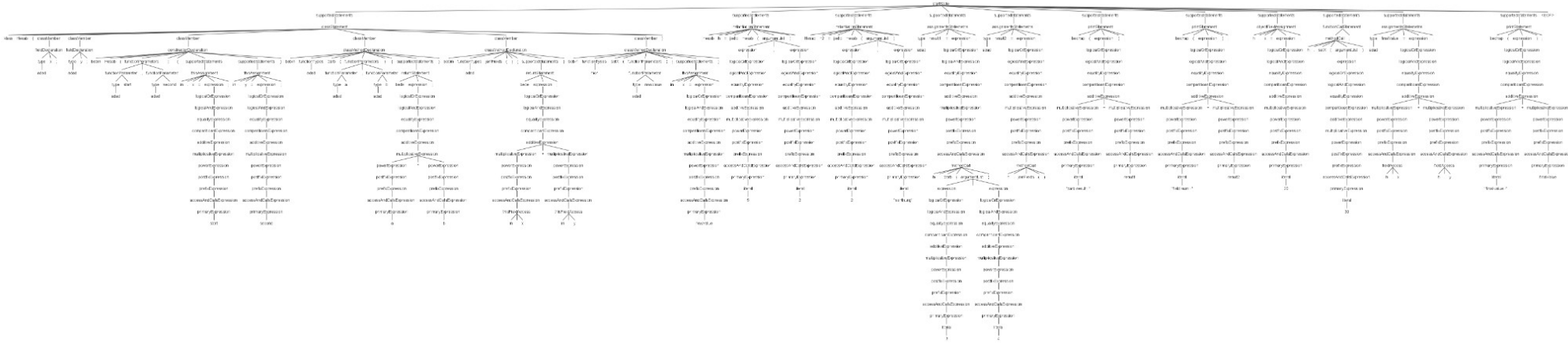
Expected behavior: Contains one intentional semantic error: a constructor argument of type matn is passed where adad is expected.

Input Test File

```
/* File_name : test_three
related contents : testing class define and instansiation
*/


klass Hesab {
    adad x;
    adad y;


    bebin Hesab(adad start, adad second) {
        in.x = start;
        in.y = second;
    }


    bebin adad zarb(adad a, adad b) {
        bede a * b;
    }

    bebin adad jamFields() {
        bede in.x + in.y;
    }

    bebin hich setX(adad newValue) {
        in.x = newValue;
    }
}

Hesab h = jadid Hesab(5, 2);
Hesab h2 = jadid Hesab(2,"sonthung"); // exception here

adad result1 = h.zarb(3, 4);
adad result2 = h.jamFields();

bechap("zarb result: " + result1);
bechap("field sum: " + result2);
```

```
h.x = 20;
h.setX(30);

adad finalValue = h.x + h.y;
bechap("final value: " + finalValue);
```

Parser Tree Image



4. Test Four - Exception Handling

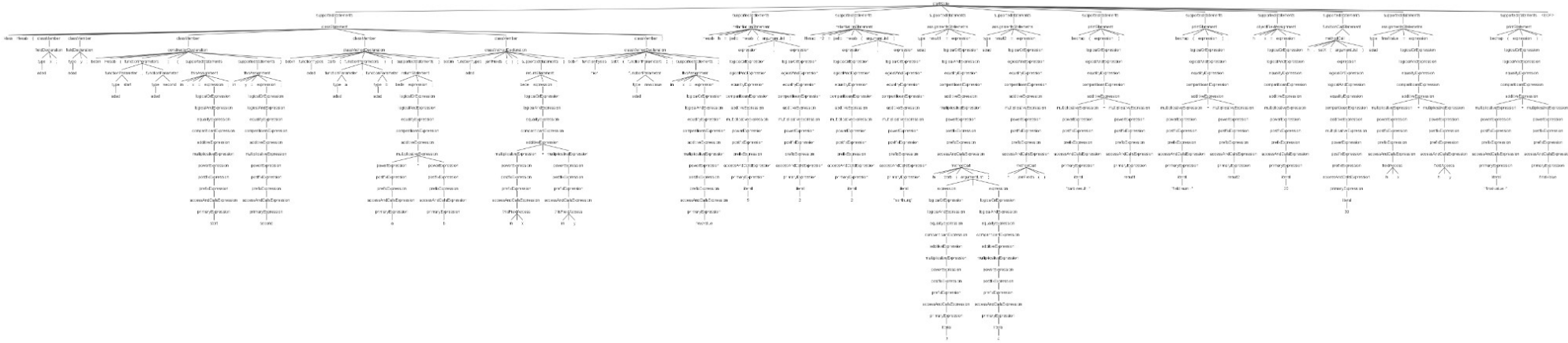
Purpose: Demonstrates custom exception declaration, try/catch/finally blocks, predefined exception types, custom exception types and exception throw syntax.

Expected behavior: Includes intentional invalid forms such as throw without parentheses and a case-sensitive keyword typo, useful for showing parser/semantic behavior.

Input Test File

```
klass MyError;


emtehan {
   bechap("inside try block");
   bendaz SefrBood();
}
gereftar (SefrBood e) {
   bechap("zero exception handled");
   bendaz SefrBood;
}
akhar {
   bechap("finally block executed");
}


emtehan {
   bendaz MyError();
}
gereftar (MyError err) {
   bechap("custom exception handled");
   bechap(err.getMessage());
   Bendaz MyError;
}
akhar {
   bechap("custom finally done");
}
```

Parser Tree Image



5. Test Five - Loops, Break and Continue

Purpose: Demonstrates while, for, nested control flow, if conditions, shekan, edame, assignment updates and print statements.

Expected behavior: Expected to pass semantic checks because loop conditions are boolean and shekan/edame are inside loops.

Input Test File

```
adad total = 0;
adad outer = 0;

ta (outer < 3) {
   bechap("outer: " + outer);

   baraye (adad i = 0; i < 10; i = i + 1) {
      age (i == 2) bood {
         edame;
      }

      age (i == 7) bood {
         shekan;
      }

      total = total + i;
      bechap("i: " + i);
   }

   age (total > 20) bood {
      bechap("total is big");
   } vagarna {
      bechap("total is small");
   }

   outer = outer + 1;
}

bechap("final total:");
bechap(total);
```

Parser Tree Image